Fortran 90 — Reference

Fortran 90 here is a free-form subset compiled with `lex` + `yacc` + `cc` (`fortran.l` / `fortran.y` lower Fortran to C; `cc` lowers the C to .NET IL). A `PROGRAM` becomes `main`; every `SUBROUTINE` / `FUNCTION` becomes a `public static` method on `CProgram`, so **C# and VB.NET can call compiled Fortran directly**.

Free-form (the layout Fortran 90 introduced) is what makes Fortran tractable for lex/yacc: whitespace is significant, so the fixed-form `DO`-loop lexing ambiguity is gone; `!` starts a comment, `&` continues a line.

```
fortran prog.f90                # compile to prog.exe (native .NET executable)
fortran prog.f90 -o app.exe     # choose the output name
fortran lib.f90 --dll           # compile to a library for C#/VB.NET to reference
```

Quick Reference

```
program name / end program        subroutine s(a) / end subroutine   ! by reference
integer function f(x) / end function   ! by value, assign result to the function name
integer :: i, n        real :: x, y     logical :: flag    character(len=20) :: s
integer :: a(10)       real :: m(3,3)   integer :: v(0:9)  ! 1-based unless lo:hi given
a = b + c*d - e/f      x = 2.0 ** n      s = "ab" // "cd"   ! ** power, // concat
if (c) then ... else if (c) then ... else ... end if
do i = 1, n[, step] ... end do          do while (c) ... end do      exit / cycle
select case (e); case (1); ... ; case (2,3); ... ; case default; ... ; end select
print *, a, b          read *, x         call s(arg)
.and. .or. .not.  .eq. .ne. .lt. .le. .gt. .ge.   ==  ≠  <  ≤  >  ≥   .true. .false.
mod abs sqrt sin cos exp log real int max min len
```

Data types

`integer` (32-bit), `real` (double precision — `double precision` is accepted as a synonym), `logical` (`.true.` / `.false.`), and `character(len=N)` (a fixed-length string). Integer and real mix in arithmetic with the usual promotion to real.

Statements / Commands

Assignment (`x = expr`), `print *,` / `read *,`, the block forms `if/else if/else/end if`, `do` / `do while` / `end do` (with `exit` and `cycle`), `select case`, `call`, and `return` / `stop`. Statements end at the newline (or `;`); `&` continues a line.

Functions

Two kinds of subprogram, both callable from C#/VB.NET as `CProgram.f_<name>`:

- `FUNCTION` returns a value; you assign the result to the function's own name. Arguments are passed **by value**, giving clean interop signatures (`int f_add(int,int)`, `double f_circle_area(double)`).
- `SUBROUTINE` returns nothing; it communicates through its arguments, which are passed **by reference** (Fortran semantics — `call swap(x, y)` modifies `x` and `y`).

Intrinsics: `mod abs sqrt sin cos exp log real int max min len`.

Input / Output

`print *, list` writes the items separated by spaces and a trailing newline (reals with `%g`, logicals as `T` / `F`). `read *, vars` reads values into variables. (List-directed I/O only; no `format` statements — see *Subset boundaries*.)

Graphics

None — Fortran is numeric/array oriented. Activity 8 below builds a **text-art** triangle by string concatenation.

Notes

- **Free-form only.** `!` comments, `&` continuation, `;` separator; columns don't matter.
- **Keywords are reserved** in this subset (you can't name a variable `if` or `real`), which avoids Fortran's classic non-reserved-keyword ambiguity. The one needed exception, `real(x)` as a type conversion, is handled specially.
- Arrays are **1-based** by default; `(lo:hi)` sets explicit bounds. Dimensions are integer literals.
- `**` is right-associative and binds tighter than unary minus (`-2**2 = -4`).

Subset boundaries

A solid free-form core, not the whole language. Not included: fixed-form source; `module` / `contains` / `use`; derived types; `allocatable` / `pointer`; array sections (`a(1:5)`) and array-valued expressions; `where` / `forall`; `format` statements; parameter-sized arrays. Subprograms are top-level (external) rather than module procedures.

Tutorial

Every example was compiled and run with `fortran`; the output shown is real.

1. Your first program

```
program t1
  implicit none
  integer :: i, n, total
  real :: x
  n = 5
  total = 0
  do i = 1, n
    total = total + i
  end do
  print *, "sum 1..", n, "=", total
  x = sqrt(2.0)
  print *, "sqrt(2) =", x
  if (total > 10) then
    print *, "big"
  else
    print *, "small"
  end if
end program t1
```

```
sum 1.. 5= 15
sqrt(2) = 1.41421
big
```

A `do` loop sums 1..5, `sqrt` is an intrinsic, and `if/else` chooses a branch.

2. Variables and data types

Variables are **declared** with a type and `::`. `implicit none` (recommended) turns off Fortran's old implicit typing so every name must be declared:

```
integer :: count
real :: x, y
logical :: ready
character(len=20) :: name
```

`integer` is a 32-bit int, `real` is double precision, `logical` holds `.true.` / `.false.`, and `character(len=N)` is a fixed-length string. Mixed arithmetic promotes to real (`5 / 2.0` is real division).

3. Flow control

Block `if`, the multi-way `select case`, and both loop forms:

```
do i = 1, 4
  select case (i)
    case (1)
      print *, "one"
    case (2, 3)
      print *, "two or three"
    case default
      print *, "other"
  end select
end do
i = 1
do while (i ≤ 3)
  print *, "count", i
  i = i + 1
end do
```

```
one
two or three
two or three
other
count 1
count 2
count 3
```

`case (2, 3)` matches either value; `do while` loops on a condition; `exit` and `cycle` break and continue.

4. Arrays

Arrays are **1-based** (or `(lo:hi)`), indexed with parentheses:

```
integer :: a(5), i
do i = 1, 5
  a(i) = i * i
end do
print *, "squares:", a(1), a(2), a(3), a(4), a(5)
```

```
squares: 1 4 9 16 25
```

`real :: m(3,3)` declares a 2-D array indexed `m(i,j)`. Dimensions are integer literals (no parameter-sized arrays in this subset).

5. Subroutines and functions

A **function** returns a value (assigned to its own name) and takes arguments **by value**; a **subroutine** returns through its arguments, passed **by reference**:

```
integer function fact(n)
  integer :: n, r, k
  r = 1
  do k = 2, n
    r = r * k
  end do
  fact = r
end function fact

subroutine swap(a, b)
  integer :: a, b, t
  t = a
  a = b
  b = t
end subroutine swap
```

Calling `print *, fact(5)` gives `120`; after `call swap(x, y)` with `x=10, y=20`, the caller's variables are exchanged:

```
fact(5) = 120
after swap: 20 10
```

That by-reference `swap` actually mutating the caller's `x` and `y` is the defining Fortran behaviour — a literal argument (`call greet(3)`) is instead staged in a temporary, since you can't take the address of a constant.

6. Memory management

There is no manual memory in this subset — storage is **static**:

- Scalars and arrays are declared with a fixed size and live for the lifetime of the program unit (`integer :: a(5)` reserves five ints).
- `character(len=N)` reserves an `N`-character buffer.
- Function/subroutine locals exist for the duration of the call.

Nothing is allocated or freed by hand (`allocatable` / `pointer` are out of scope — see *Subset boundaries*).

7. Strings and a text layout

`character(len=N)` strings are joined with `//`:

```
character(len=20) :: msg
msg = "Fortran" // " 90"
print *, msg
```

```
Fortran 90
```

`len(s)` gives a string's length. (String handling is deliberately small; there is no substring/`index`/`trim` library here.)

8. Drawing a picture

No graphics, so the nearest thing is **text art** — build each row by concatenation and print it:

```
program tri
  implicit none
  integer :: i, j
  character(len=20) :: row
  do i = 1, 5
    row = ""
    do j = 1, i
      row = row // "*"
    end do
    print *, row
  end do
end program tri
```

```
*
**
***
****
*****
```

The inner loop appends a `*` per column; the outer loop prints rows of growing length.

9. Calling Fortran from C# / VB.NET

This is the payoff of compiling to real .NET IL. Compile a Fortran file as a library:

```
integer function add(a, b)
  integer :: a, b
  add = a + b
end function add

real function circle_area(r)
  real :: r
  circle_area = 3.14159265 * r * r
end function circle_area
```

```
fortran lib.f90 --dll          # → fortranlib.dll
```

Each function lands on `CProgram` as a `public static` method (prefixed `f_`). A C# program that references `fortranlib.dll` (and `CRuntime.dll`) calls them like any API:

```
Console.WriteLine($"add(20, 22)        = {CProgram.f_add(20, 22)}");
Console.WriteLine($"fib(15)            = {CProgram.f_fib(15)}");
Console.WriteLine($"circle_area(3.0)   = {CProgram.f_circle_area(3.0)}");
```

```
add(20, 22)        = 42
fib(15)            = 610
circle_area(3.0)   = 28.274333850000005
```

Because functions pass arguments by value, the signatures are ordinary .NET ones (`int f_add(int, int)` , `double f_circle_area(double)`) — full type checking on the C# side. From here, grow the subset toward modules and array-valued expressions (*Subset boundaries*), or use it as-is for numeric kernels that C#/VB.NET drive.